

Reviewer Pack — Real Algebraic Geometry of Polynomial Systems Bounds DPLL Re...

Entry 1fe59524b206 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 01:53:35 UTC

Real Algebraic Geometry of Polynomial Systems Bounds DPLL Refutation Size

Entry ID: 1fe59524b206 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 01:53:35 UTC

1. Conjecture

Field A (mathematical branch): Real algebraic geometry **Field B** (complexity object): Complexity Theory: DPLL refutation size

Statement:

{‘quantitative’: ‘For any 3-CNF formula F with n variables, let A_F be the ideal generated by the equations of its unsatisfiability. Then, the number of distinct real points in the variety defined by A_F is lower-bounded by a polynomial function in n .’, ‘falsifier’: ‘There exists a 3-CNF formula F such that the variety defined by the ideal A_F has fewer than a constant times n real points.’}

Rationale (proposer’s reasoning):

{‘explanation’: ‘If real algebraic geometry can be used to bound the number of real points in the variety defining unsatisfiability, it may provide insights into the structure of the DPLL search tree that are not apparent through other complexity-theoretic methods.’, ‘linkage’: ‘This conjecture bridges real algebraic geometry with complexity theory by proposing a quantitative relationship between real points in varieties and the complexity of solving Boolean satisfiability problems.’}

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 70d08bade9f91378

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a 3-CNF formula F with n variables, if the number of distinct real points in the variety defined by the ideal A_F is at least cn for some constant c and all seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'real algebraic geometry' AND 'DPLL refutation size' AND polynomial - 'variety of ideals' AND 'lower bound' AND 'real algebraic geometry' - 'complexity theory' AND 'DPLL refutation size' AND 'real points in varieties'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_3cnf(n):
    literals = [f'x{i}' for i in range(1, n+1)] + [f'~x{i}' for i in range(1, n+1)]
    clauses = []
    for _ in range(2*n):
        clause = random.sample(literals, 3)
        if random.choice([True, False]):
            clause[0] = '~' + clause[0]
            clause[1] = '~' + clause[1]
            clause[2] = '~' + clause[2]
        clauses.append(' & '.join(clause))
    return ' | '.join(clauses)

def solve_system(literals, clauses):
    n = len(literals) // 2
    A = [[0 for _ in range(n)] for _ in range(n)]
    b = [0] * n

    for clause in clauses:
        literals_in_clause = set()
        for literal in clause.split(' & '):
            if literal.startswith('~'):
                literals_in_clause.add(literal[1:])
            else:
                literals_in_clause.add(literal)

    for lit1 in literals_in_clause:
```

```

        i = literals.index(lit1) // 2
        A[i][i] += 1

    for lit1, lit2 in itertools.combinations(literals_in_clause, 2):
        i = literals.index(lit1) // 2
        j = literals.index(lit2) // 2
        if i != j:
            A[i][j] -= Fraction(1, 2)
            A[j][i] -= Fraction(1, 2)

    for i in range(n):
        b[i] = Fraction(1, 2)

# Gaussian elimination with partial pivoting
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]

# Back substitution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]

# Count distinct real points
    num_real_points = 0
    for i in range(n):
        if abs(x[i]) < 1e-6:
            continue
        num_real_points += 1

    return num_real_points

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        formula = generate_3cnf(n)
        literals = list(set(literal.strip('~') for literal in formula.split(' | ')))
        num_real_points = solve_system(literals, formula.split(' | '))

        result = {
            "metric_name": "num_real_points",
            "metric_value": num_real_points,

```

```

        "instances_tested": 1,
        "conjecture_holds": None,
        "counterexample": ""
    }

    if n == 5:
        result["conjecture_holds"] = num_real_points >= 2
    elif n == 10:
        result["conjecture_holds"] = num_real_points >= 4
    elif n == 15:
        result["conjecture_holds"] = num_real_points >= 6
    elif n == 20:
        result["conjecture_holds"] = num_real_points >= 8
    elif n == 30:
        result["conjecture_holds"] = num_real_points >= 10
    elif n == 40:
        result["conjecture_holds"] = num_real_points >= 12

    results.append(result)

total_metric_value = sum(result["metric_value"] for result in results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "metric_name": "num_real_points",
    "mean_metric_value": total_metric_value / len(results),
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

    # Compute mean/std of metric_value and fraction of seeds where conjecture_holds
    total_metric_values = [result["mean_metric_value"] for result in seeds]
    support_fractions = [result["support_fraction"] for result in seeds]

    mean_metric_value = sum(total_metric_values) / len(seeds)
    std_metric_value = math.sqrt(sum((x - mean_metric_value) ** 2 for x in total_metric_values) / len(seeds))
    support_fraction = sum(support_fractions) / len(seeds)

    if all(result["conjecture_holds"] for result in seeds):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in seeds):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07c3966b.py", line 135, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07c3966b.py", line 95, in run_trial
    num_real_points = solve_system(literals, formula.split(' | '))
                   ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07c3966b.py", line 44, in solve_system
    i = literals.index(lit1) // 2
       ~~~~~^~~~~~
ValueError: 'x3' is not in list
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and provide results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11358
2	preregistration	ollama_remote	glm4:latest	0	0	5228
3	novelty	ollama_remote	glm4:latest	0	0	4779
4	novelty	ollama_remote	glm4:latest	0	0	5471
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14436
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12849
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12227
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16352
9	judge	ollama_remote	glm4:latest	0	0	8728

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 91428 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1fe59524b206.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1fe59524b206.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1fe59524b206.tar.gz` (if generated)